CTSE & LOB — Complete Build Guide

Nov 5, 2026 · @Naman Jain

How to use this guide

This is the reasoning behind every design decision in both projects, written so you can rebuild them yourself and defend each choice under questioning.

The two projects answer different kinds of question and are deliberately different in character.

	CTSE	LOB
Kind	Controlled experiment	Systems build
Question	How much does dispatch cost, and why?	How do you build a correct, fast order book?
Input	524,288 synthetic ticks in memory	60 MB binary ITCH file
Output	Cycle counts and assembly listings	Pass/fail gates, throughput tables
Proves	Measurement rigour, compiler behaviour	Correctness discipline, wire formats
Lines	~2,900	~3,750

Keep them as two repositories. A reviewer skims — "dispatch benchmark" and "ITCH order book" each land in five seconds; a merged "trading system" invites the question of which part is actually yours.

Read the CTSE half first even if the order book interests you more. CTSE establishes the measurement methodology that LOB reuses, and the rules about what makes a benchmark trustworthy apply to both.

Three conventions used throughout:

- **Cycles, not nanoseconds**, for anything small. Cycle counts survive a change of clock speed; nanoseconds do not.
- **Minimum, not mean.** On a machine you do not fully control, every sample is the true cost plus a non-negative interference term. The minimum estimates the truth; the mean estimates the truth plus your neighbours' behaviour.
- **Every number here was measured**, on a virtualised Xeon at 2.80 GHz with GCC 13.3. Your hardware will differ. Re-run everything and replace these tables with your own figures — numbers you did not measure are numbers you cannot defend.

CTSE: the question

You have strategy code that runs on every market tick. There are two ways to write it, and the folklore says one is faster. The project measures by how much, and — more usefully — *why*.

Way A, runtime dispatch. Strategies are objects behind an interface. You loop over a container and call `on_tick()` on each. Strategies can be chosen at startup from a config file, swapped, loaded from a shared object.

Way B, compile-time dispatch. The strategy set is baked into the type via CRTP and templates. No runtime flexibility, but the compiler can see everything.

The usual answer — "virtual is slow because of the vtable lookup and branch misprediction" — turns out to be wrong in an interesting way, and that is the finding the project exists to produce.

Why this needs an experiment

Three claims float around, all stated confidently, none of them checkable from a blog post:

1. Virtual dispatch costs a cache miss for the vptr load.
2. Virtual dispatch costs a branch misprediction on the indirect call.
3. Templates are faster because they avoid the `call` instruction.

All three are either wrong or heavily conditional. The vptr is almost always hot. The misprediction only materialises under conditions most code never meets. And the `call` instruction is nearly free — it is what the call *prevents* that costs.

You cannot settle this by reading. You settle it by building two versions that provably compute the same thing, timing them under controlled conditions, and reading the assembly to explain the gap.

What the project is not

It is not a trading system. It generates fake ticks, computes signals, and prints cycle counts. Nothing is traded, no orders exist, the strategies are textbook indicators chosen because they are recognisable and have realistic arithmetic — not because they make money.

That framing matters in an interview. The deliverable is a *measurement*, and the credibility comes from the controls, not the speedup.

CTSE: shared cores — the decision the whole project rests on

The arithmetic for every strategy is written **exactly once**, in `cores.hpp`. Nothing else in the project computes anything.

```
template <std::size_t Fast, std::size_t Slow, int ThreshBps>
struct EmaCrossCore {
    CTSE_ALWAYS_INLINE Signal step(const Tick& t) noexcept {
        const double m = t.mid();
        fast_.push(m);
        slow_.push(m);
        if (!slow_.warm()) [[unlikely]] return no_signal();
        const double d = (fast_.value() - slow_.value()) / m;
        return to_signal(d / kThresh, 1.0, kId);
    }
    ind::Ema<Fast> fast_{};
    ind::Ema<Slow> slow_{};
};
```

The static arm is a CRTP shell around this core. The virtual arm is a vtable-bearing shell around the *same* core. Both call `core.step(tick)`. The arms differ only in how `step` is reached.

The failure mode this prevents

The usual way this benchmark goes wrong is subtle enough that people publish it without noticing. Someone writes the virtual class with a `double alpha_` member and the template class with a `static constexpr kAlpha`, then reports a 4x speedup and attributes it to dispatch.

Most of that 4x was constant folding of `alpha`. The template version had its coefficient as a compile-time immediate the compiler could fold into an FMA; the virtual version loaded it from memory every tick. That is a real effect, but it is not dispatch, and reporting it as dispatch is wrong.

Sharing the cores makes that mistake structurally impossible. There is no second copy of the arithmetic to accidentally differ.

Why the cores are still templated

The cores take window sizes as non-type template parameters, so both arms get compile-time-sized ring buffers and folded EMA coefficients. That looks like it contaminates the comparison — but it cancels, because **both arms get it equally**.

What does not cancel is inlining, and that is precisely the effect under test. Keeping the NTTP sizing in the shared core is how you isolate inlining from every other compile-time benefit.

The gate that enforces it

`make verify` runs both arms over the same tick stream and asserts the emitted signal streams are bit-identical, including the IEEE-754 bit pattern of every `double`. Not "within tolerance" — tolerance would hide exactly the structural divergences the test exists to catch.

If that assertion fails, the benchmark is comparing two different computations and its numbers mean nothing. You run it before you are allowed to believe any timing result.

CTSE: types.hpp and data.hpp

Tick is exactly one cache line

```
struct alignas(64) Tick {
    std::uint64_t ts_ns;
    std::uint32_t instrument_id;
    std::uint32_t seq;
    double        bid, ask, last;
    std::uint32_t bid_qty, ask_qty, last_qty, flags;
};
static_assert(sizeof(Tick) == 64);
static_assert(alignof(Tick) == 64);
```

64 bytes, 64-byte aligned, asserted at compile time. Two reasons:

No straddling. A tick that spans two cache lines costs two L1 fills instead of one on every access. In an array of ticks walked linearly, an unaligned 56-byte struct would have roughly one in eight ticks straddling — a 12% penalty for nothing.

Clean prefetch stride. The hardware prefetcher recognises unit-stride access. A 64-byte aligned array gives it exactly that.

The `static_assert`s matter more than the `alignas`. If someone adds a field later, the build fails loudly instead of silently losing the property.

Signal is 16 bytes so it returns in registers

```
struct Signal {
    double        strength;
    Side          side;
    std::uint8_t  confidence;
    std::uint16_t strategy_id;
    std::uint32_t _pad;
};
static_assert(sizeof(Signal) == 16);
```

Under the System V ABI a 16-byte trivially-copyable aggregate is returned in registers. Make it 17 bytes and the compiler switches to a hidden pointer (sret): the caller allocates stack space, passes its address, the callee writes through it. That is a store and a load on every strategy call, and it would show up in both arms — but unequally, because the inlined arm can often eliminate it and the opaque arm cannot.

The explicit `_pad` is there so the padding is visible and deliberate rather than something the compiler inserted that you then forget about.

The generator is deterministic by construction

`data.hpp` uses xoshiro256** with a fixed seed, not `std::mt19937`. Two reasons: mt19937 is slower and, more importantly, the standard's distribution adapters are **not** specified to produce identical output across implementations. `std::uniform_real_distribution` can legally differ between libstdc++ and libc++. A benchmark whose input changes when you switch compilers is not reproducible.

The generator is not trying to be a realistic price model. It is trying to **exercise every branch at a realistic rate**:

- GBM-ish random walk so EMAs cross at a sane frequency
- occasional volatility bursts, so the z-score strategy actually fires
- spread that widens with volatility, so the spread gate flips
- skewed queue sizes with autocorrelation, so imbalance is not always near zero

If a branch never flips during the run, you are measuring a branch predictor at 100% accuracy and the numbers will not survive contact with real data. `verify` prints the signal activity rate and warns if it falls outside 1–99%. Measured: **77.35% of ticks fire**, roughly balanced between buy and sell.

CTSE: clock.hpp — timing without lying to yourself

Why not std::chrono

`clock_gettime` through the vDSO costs roughly 20–25 ns. The thing being measured is 8 ns. The instrument would dominate the measurement by 3x.

`RDTSC` costs 6–8 cycles, `RDTSCP` 24–32. Both are far cheaper and, critically, *stable* enough that the overhead can be measured once and subtracted.

Fence placement, precisely

RDTSC is not ordered against surrounding instructions. Out-of-order execution can hoist it above the code under test or sink it below. The fencing is asymmetric and each half does a different job:

```
[[gnu::always_inline]] inline std::uint64_t tsc_begin() noexcept {
    _mm_lfence();            // nothing before can sink past this
    return __rdtsc();
}

[[gnu::always_inline]] inline std::uint64_t tsc_end() noexcept {
    unsigned aux;
    const std::uint64_t t = __rdtscp(&aux);   // waits for prior ops to
retire
    _mm_lfence();                              // nothing after can hoist
above
    return t;
}
```

- **Opening**: `LFENCE` then `RDTSC`. The fence stops earlier instructions sinking into the timed region.
- **Closing**: `RDTSCP` then `LFENCE`. `RDTSCP` itself waits for previously issued instructions to retire; the `LFENCE` stops later instructions being hoisted above it.

`CPUID` + `RDTSC` is the older recipe you will find in textbooks. **Do not use it on a VM.** `CPUID` traps to the hypervisor on most virtualised hosts, costing hundreds of cycles with terrible variance. `LFENCE` is correct on anything post-Nehalem.

The precondition people skip

```
grep -o 'constant_tsc\|nonstop_tsc' /proc/cpuinfo | sort -u   # must show
both
```

Without `constant_tsc` the TSC changes rate with P-states, and without `nonstop_tsc` it halts in deep C-states. Either way every number you produce is fiction. Check before you measure, not after someone questions your results.

Calibration and overhead

`calibrate_tsc_ghz()` measures TSC ticks against `CLOCK_MONOTONIC` over a 60 ms window, five times, and takes the **median** — long enough to swamp syscall jitter, median to reject one bad sample.

`measure_timer_overhead()` runs 20,000 empty `tsc_begin`/`tsc_end` pairs and returns the **minimum**. The minimum is the cleanest estimate of the instrument's own cost, since every sample is that cost plus non-negative noise.

That overhead is subtracted in Experiment E, where per-operation timing makes it significant. It is *not* subtracted in the throughput experiments, where one timer pair wraps 524,288 ticks and the overhead is below 0.01% of the measurement.

CTSE: bench.hpp — the three ways a microbenchmark lies

1. Dead-code elimination

If nothing observes the result, the compiler deletes the work and you measure an empty loop.

```
template <class T>
[[gnu::always_inline]] inline void sink(const T& v) noexcept {
    asm volatile("" : : "r,m"(v) : "memory");
}
```

An empty inline-asm block that claims to consume `v`. No instruction is emitted. The `"r,m"` constraint lets the compiler keep the value in a register if it can and spill to memory otherwise — `"m"` alone would *force* a store, which is a real cost you did not intend to add.

This is the detail to be able to explain. A reviewer who knows microbenchmarking will ask why not `"m"`.

2. Loop-invariant hoisting

If the input is a compile-time constant, the whole computation folds once and the loop measures nothing. Fixed by reading inputs from a heap array built at runtime.

3. Drift between arms

Turbo ramp-up, thermal throttling, and the scheduler make whichever arm runs first systematically different from the one that runs last.

Fixed by running arms **round-robin**: A B C, A B C, A B C — never A×25 then B×25. Drift becomes common-mode and cancels in the comparison. Each round also constructs a fresh engine, so both arms see identical warm-up state.

The anti-DCE choice that matters

Inside the timed loop there is a loop-carried checksum, **not** a per-iteration memory clobber:

```
for (std::size_t i = 0; i < n; ++i) {
    const Signal s = eng.on_tick(p[i]);
    std::uint64_t b;
    std::memcpy(&b, &s.strength, sizeof(b));
    chk = chk * 1099511628211ULL ^ (b + static_cast<std::uint64_t>(s.side));
}
sink(chk);
```

A `clobber()` inside the loop would force all engine state out of registers and back through memory every iteration. That penalises the **static arm specifically** — the arm whose advantage is keeping state in registers across iterations — which is exactly the effect under test. One `sink` after the loop is sufficient and symmetric.

This is the kind of mistake that produces a benchmark showing no difference, which the author then publishes as "templates don't actually help."

Reporting: never the mean

A mean over a latency distribution with a fat tail is a number that describes no observation you actually made. `Stats::from` reports min, p50, p90, p99, p99.9, max.

- **Minimum** answers "what does this code cost when nothing interferes."
- **p99** answers "what will production see."

They are different questions. On a shared or virtualised host the minimum is the only number to trust; on a tuned box with `isolcpus` and `nohz_full`, p99 becomes meaningful too.

`pin_to_core` returns `bool` and the harness **says so loudly** when pinning fails, rather than silently reporting noisy numbers. Containers often forbid it.

CTSE: indicators.hpp — why window sizes are template parameters

Every window size is a non-type template parameter, not a constructor argument. That is the point of the exercise, and it buys three things a runtime size cannot.

1. State lives inline

```
template <class T, std::size_t N>
class RollingWindow {
    static_assert(is_pow2(N), "N must be a power of two so % becomes &");
    std::array<T, N> buf_{};   // not a pointer — the storage IS the member
    T                sum_{};
    std::uint64_t    head_{0};
};
```

The ring buffer is a fixed-size array member. No pointer indirection, no separate allocation, and the whole strategy object can fit in a cache line or two. A runtime-sized version needs a heap buffer and a dependent load on every access.

2. Modulo becomes a mask

`i & (N-1)` is one `AND` instruction. `i % n` with runtime `n` is a 20–40 cycle integer divide. The `static_assert` on power-of-two is what makes the compiler able to do this — without it, even a compile-time `N` of 100 produces a multiply-shift sequence rather than a single mask.

This is why the assert is phrased with its reason in the message. A future reader who wants `N = 100` should see immediately why they cannot have it.

3. Coefficients fold into immediates

```
template <std::size_t Period>
class Ema {
public:
    static constexpr double kAlpha = 2.0 / (static_cast<double>(Period) +
1.0);
    CTSE_ALWAYS_INLINE void push(double x) noexcept {
        value_ += kAlpha * (x - value_);   // one FMA once alpha is an
immediate
    }
};
```

With `kAlpha` a compile-time constant and `-march=native`, this becomes a single `vfmadd`. With a runtime `alpha_` member it is a load, a subtract, a multiply, an add.

The cost, stated honestly

A distinct instantiation per parameter set. Binary size and compile time grow. Eight strategies across six parameter sets is 48 copies of the code competing for L1 instruction cache.

That trade is correct for a handful of hot strategies and **wrong for a config-driven system with hundreds of variants**. Say so in an interview — the trade-off is the answer, not the technique.

The numerical honesty you should volunteer

Incremental float sums drift. `sum_ += v - evicted` is not the exact inverse of the earlier `sum_ += v`, and over millions of ticks the error accumulates.

The code does a periodic full recompute every 2^{20} pushes, marked `[[unlikely]]` and `CTSE_NOINLINE` so it stays out of the hot path's instruction footprint. That is the cheap mitigation. The better ones, in order of cost:

1. Periodic recompute (what this does)
2. Kahan or Neumaier compensated summation
3. **Integer fixed-point accumulation** — exactly reversible, and what production systems actually use for prices

Knowing that option 3 is the real answer, and that this code took option 1 deliberately, is worth more than having implemented option 3.

CTSE: static_engine.hpp — CRTP, concepts, tuple chains

The CRTP mechanism

```
template <class Derived>
struct StrategyBase {
    CTSE_ALWAYS_INLINE Signal on_tick(const Tick& t) noexcept {
        return static_cast<Derived*>(this)->on_tick_impl(t);
    }
};

struct EmaCross : StrategyBase<EmaCross> {
    CTSE_ALWAYS_INLINE Signal on_tick_impl(const Tick& t) noexcept {
        return core_.step(t);
    }
};
```

The base knows its derived type as a template parameter, so `static_cast<Derived*>(this)` is resolved at compile time and `on_tick_impl` is a direct, inlinable call. Static polymorphism: an interface contract and code reuse with zero runtime indirection.

The class is its own template argument — that is the "curiously recurring" part, and it is legal because at the point `StrategyBase<EmaCross>` is instantiated, `EmaCross` is incomplete but the base only needs its *name*, not its layout. The `static_cast` happens inside a member function body, which is instantiated later, when `EmaCross` is complete.

Why CRTP rather than free functions

CRTP lets the base implement **shared behaviour in terms of derived hooks**:

```
CTSE_ALWAYS_INLINE Signal on_tick_counted(const Tick& t) noexcept {
    ++ticks_seen_;
    const Signal s = on_tick(t);
    fires_ += static_cast<std::uint64_t>(s.active());
    return s;
}
```

Every strategy inherits tick counting and fire counting without writing it, and without a call. Measured cost: **zero** (21.62 vs 21.73 cycles — noise). That is the thing plain function templates give you less cleanly.

Concepts for diagnostics

```
template <class S>
concept Strategy = requires(S s, const Tick& t) {
    { s.on_tick(t) } noexcept -> std::same_as<Signal>;
    { S::id() } -> std::convertible_to<std::uint16_t>;
};
```

CRTP for the mechanism, concepts for the error messages. A malformed strategy fails at the definition site with one readable line instead of forty lines of template instantiation spew. Use both — this is the single biggest usability improvement C++20 brought to this pattern.

The chain: a tuple, not a vector

```
template <Strategy... S>
class Chain {
    std::tuple<S...> legs_{};
public:
    CTSE_ALWAYS_INLINE Signal on_tick(const Tick& t) noexcept {
        Signal acc = no_signal();
        std::apply([&](auto&... s) {
            ((acc = combine(acc, s.on_tick(t))), ...);
        }, legs_);
        return acc;
    }
};
```

The strategies live in a `std::tuple`, so they are laid out contiguously **inside** the chain object — one allocation, no pointer chasing. `std::apply` plus a comma fold expands to a straight-line sequence of inlined bodies.

There is no loop over strategies at runtime. The loop was unrolled by the type system.

if constexpr vs a runtime flag

```
if constexpr (F.use_imbalance) acc = combine(acc, imb_.on_tick(t));
```

`if constexpr` deletes the untaken branch entirely: no test, no jump, no instruction-cache footprint, and the dead code's state is never initialised. A runtime `if (flag)` must keep both paths resident and load the flag.

Note the honest limitation in the code: the members exist unconditionally for layout simplicity, so `if constexpr` removes the *code paths* but not the storage. Dropping the storage too would need specialisation on `F`, or `[[no_unique_address]]` with an empty

tag type. Worth knowing you did not do it, and why.

CTSE: virtual_engine.hpp — three arms, each isolating one thing

"Virtual is slow" is a lazy claim. The interesting question is *which part* is slow, so there are three virtual arms rather than one.

Arm	Setup	What it isolates
Opaque	Concrete classes defined only in `virtual_impl.cpp`, no LTO	The realistic case: engine and strategy in different TUs, optimiser blind
Visible + `final`	Concrete classes in the header, marked `final`	How much the compiler claws back when it *can* see everything
Arena	Placement-new into one contiguous 64-byte-aligned buffer	Separates dispatch cost from allocation-layout cost

Including the `final` arm is the honest move. It shows the enemy is the optimisation barrier, not the vtable — and that result is more useful than the headline speedup.

Anatomy of the cost, in order of magnitude

1. The inlining barrier — dominant, 70–90% of the delta.

An opaque call forces the ABI: arguments into specific registers, caller-saved registers spilled to the stack, no constant propagation into the body, no common-subexpression elimination across strategies (every leg recomputes `t.mid()`), no vectorisation across legs.

2. The indirect branch — conditional.

Load vptr, load slot, `call *`. If one type dominates, the branch target buffer predicts it and this is 2–3 cycles. With several types interleaved unpredictably it mispredicts and costs 15–20. Experiment B shows this only materialises under *random* ordering.

3. Memory layout — often zero.

`vector<unique_ptr<IStrategy>>` is a pointer array to scattered heap objects. Each leg is a dependent load chain: pointer → object → vptr → vtable → function. Four serialised loads before the first instruction of the strategy runs. Measured at **zero** in this workload, because four legs fit comfortably in L1.

The clone types — the control that makes Experiment B interpretable

```
template <int Tag>
class OpaqueEmaClone : public IStrategy {
    Signal on_tick(const Tick& t) noexcept override { return core_.step(t); }
    core::EmaCrossCore<8, 64, 5> core_{};   // IDENTICAL arithmetic
};
```

Four distinct dynamic types wrapping byte-for-byte identical arithmetic. They differ **only** in vtable address.

Without this, "monomorphic vs polymorphic" secretly compares different amounts of floating-point work. My first attempt at Experiment B did exactly that and produced an incoherent result — monomorphic virtual appeared *slower* than polymorphic — because the mixed chain contained two cheap legs and simply did less work.

These clones require `-fno-ipa-icf`. See the traps section; this is the single most important build flag in the project.

CTSE: verify_main.cpp — the gate

Runs every arm over the identical tick stream and asserts the emitted `Signal` streams are **bit-identical**, including the IEEE-754 bit pattern of `strength`.

```
std::uint64_t bits(double d) noexcept {
    std::uint64_t u;
    std::memcpy(&u, &d, sizeof(u));   // not a reinterpret_cast — that's UB
    return u;
}

bool identical(const Signal& a, const Signal& b) noexcept {
    return bits(a.strength) == bits(b.strength) && a.side == b.side &&
           a.confidence == b.confidence && a.strategy_id == b.strategy_id;
}
```

Why bit-identical, not tolerance

Both arms call the same core, so **any** difference means a real structural divergence: different template arguments, different ordering in the combiner, a reassociation caused by `-ffast-math` applied unevenly.

Tolerance would hide exactly the bugs this test exists to catch. There is no legitimate reason for the two arms to differ by 1e-15, so accepting 1e-15 only buys you the ability to miss a real problem.

What it checks

Check	Why it is there
Single strategy: static vs opaque vs visible	Baseline equivalence
4-leg chain: static vs opaque vs arena	Chain traversal order matters
Clone4: static vs mono-virtual vs poly-virtual	The Experiment B control
`if constexpr` vs runtime flags	Experiment D equivalence
Chain2 plain vs instrumented	CRTP base behaviour changes nothing
Signal activity rate	The input exercises branches

That last one is not an assertion — it is a printed statistic with a warning band:

```
[INFO] signal activity: 77.35% of ticks fire (77086 buy / 77617 sell)
```

If activity fell below 1% or above 99%, every branch would be one-sided, the predictor would run at 100%, and the benchmark would measure a machine state that does not exist in production. **This counter is the thing that caught the equivalent bug in the order book project** — see the LOB failures section.

On divergence

It prints the first differing tick index, both signals in full, and both raw bit patterns, then exits non-zero. Finding the *first* divergence is most of the debugging; a checksum mismatch reported only at the end tells you nothing about where to look.

CTSE: the five experiments

Intel Xeon @ 2.80 GHz (single vCPU, virtualised), GCC 13.3, `-O2 -std=c++20`, 524,288 ticks per pass, 25 interleaved rounds + 3 warm-up. All figures **cycles per tick, lower is better**.

A — single strategy, monomorphic call site

arm	min	p50	p99	ns/tick	vs base
virtual, opaque (cross-TU)	33.40	34.25	45.86	11.93	1.00x
virtual, visible + `final`	21.55	22.66	27.46	7.70	**1.55x**
static CRTP	22.34	22.94	35.53	7.98	**1.49x**

arm	min	p50	p99	ns/tick	vs base
static CRTP + base instrumentation	21.62	21.84	28.02	7.72	1.55x

One dynamic type, so the BTB is always right and misprediction is *not* a factor. The entire ~11 cycle delta is the inlining barrier.

Two findings: **`final` ≈ CRTP** (this is the most important row in the project), and **CRTP's shared base behaviour is free**.

B — branch prediction and layout, controlled

All rows run four `EmaCrossCore<8,64,5>` legs. Identical arithmetic, identical state size.

arm	min	ns/tick
virtual x4, 1 type, fixed cycle	160.40	57.28
virtual x4, 4 types, fixed cycle	153.17	54.70
virtual x4, 1 type, **random order**	141.85	50.66
virtual x4, 4 types, **random order**	**198.45**	70.87
static x4, identical legs	69.69	24.89
virtual x4 mixed, heap layout	128.32	45.83
virtual x4 mixed, arena layout	128.94	46.05

Polymorphism in a fixed cycle costs nothing. Four distinct vtable targets visited as 0,1,2,3,0,1,2,3… are not slower than one target. A modern ITTAGE-style predictor learns that pattern perfectly. "Virtual is slow because of branch misprediction" is false for the common case of a fixed strategy chain.

Polymorphism in random order costs ~14 cycles per call.

```
198.45 − 141.85 = +56.6 cycles for 4 calls ≈ 14.2 cycles/call
```

Which matches theory: a 4-way random choice mispredicts ~75% of the time, and this core's mispredict penalty is ~19 cycles. 0.75 × 19 ≈ 14.

Heap scatter vs arena: no measurable difference. With four legs everything stays in L1 and the pointer chase is fully overlapped by out-of-order execution. Real effect, but it needs many more legs or genuine cache pressure — worth knowing before rewriting an allocator to chase it.

C — chain-length scaling

legs	virtual	static	ratio	tax total	tax/leg
1	52.20	22.98	2.27x	29.2	29.2
2	83.40	43.58	1.91x	39.8	19.9
4	130.40	82.13	1.59x	48.3	12.1
8	243.42	119.21	2.04x	124.2	15.5

Absolute tax grows monotonically. The *ratio* does not behave tidily — it dips at N=4 and recovers at N=8, because the mixed 4-leg chain includes two cheap legs that dilute per-leg dispatch cost. Reported as measured rather than smoothed into a nicer story.

D — if constexpr vs runtime flag

arm	min	ratio
runtime flags, all on	64.40	1.00x
`if constexpr`, all on	61.24	1.05x
runtime flags, only EMA	32.04	1.00x
`if constexpr`, only EMA	22.98	**1.39x**

The second pair is the interesting one. With features *off*, both versions skip the same work and every runtime branch is perfectly predicted — yet compile-time is still 39% faster. The gap is not the branch. The runtime version cannot prove the disabled sub-strategies are dead, so their state stays live and the code paths stay resident.

E — per-tick latency distribution

Fenced per operation, overhead subtracted. Cycles.

arm	min	p50	p90	p99	p99.9
virtual chain x4 (opaque)	126	144	184	306	827
static chain x4	92	102	146	220	926
virtual single (opaque)	38	46	50	106	401
static single	40	46	50	110	448

The last two rows say virtual and static are the same — and Experiment A said 1.5x. Both are correct, and the contradiction is the most useful result here.

The per-op measurement wraps every call in `LFENCE` / `RDTSCP`, forcing retirement and draining the pipeline at every tick boundary. But the static arm's advantage is largely *cross-iteration instruction-level parallelism*: with the body inlined, work from tick N+1 begins before tick N retires. The fence destroys exactly that. **The instrument removed the effect it was measuring.**

The chain rows still show the gap (126 vs 92) because there is enough intra-tick work to overlap inside one fenced region.

Practical rule: fenced per-operation timing gives honest tail latency but systematically understates throughput-oriented optimisations. Measure both, and know which question you are answering.

Cross-build comparison

arm	O2	O3	LTO	native
virtual, opaque	33.40	32.33	32.69	37.09
virtual, visible + final	21.55	21.85	22.88	27.48
static CRTP	22.34	21.48	22.65	27.73
static x4, identical legs	69.69	**41.47**	69.20	51.05

The `-O3` row is the headline: **a further 1.68x**. With four identical inlined legs, `-O3` hoists `t.mid()` as a common subexpression and vectorises the four independent EMA dependency chains into SIMD lanes.

The virtual version cannot access this at any optimisation level, because the legs are opaque functions and the compiler cannot prove they are independent — or even that they are the same function.

Note LTO did **not** recover the opaque-virtual gap here. GCC's speculative devirtualisation is conservative about a base pointer stored in a struct and returned from a factory. Verify on your own build rather than assuming.

CTSE: the three traps

These are worth more than the speedups. Anyone can quote a 1.5x number; being able to say "here is how this benchmark almost lied to me" is a different signal.

1. Identical code folding silently erases polymorphism

GCC's `-fipa-icf` is **on by default at `-O2`**. It merges functions with identical bodies. The four clone types have identical `on_tick` bodies, so ICF collapses them into one function — every vtable slot points at the same address, the indirect branch has one target, and the misprediction penalty vanishes while the code still *looks* polymorphic.

```
$ nm -C --defined-only build/virtual_impl.o | grep clone | grep on_tick
build/virtual_impl.o     -> 4 distinct addresses
build/virtual_impl_icf.o -> 1 distinct address
```

```
measured polymorphism penalty (random order, poly − mono):
  with ICF disabled (honest) :  +59.27 cycles / 4 calls
  with ICF enabled  (the lie):   −1.73 cycles / 4 calls
```

A real 59-cycle effect, gone. `make icf` reproduces it on demand.

I hit this in my own Makefile. The `lto` and `native` targets were initially missing `-fno-ipa-icf` and reported no polymorphism penalty at all. The build succeeded, the tests passed, and the numbers were wrong.

The general lesson: a compiler flag you did not know you were setting can delete the effect you are measuring. Check the symbols, not just the output.

2. The instrument can destroy the effect

Experiment E, covered above. Fenced per-operation timing serialises the pipeline it is trying to measure, and therefore understates any optimisation whose benefit is cross-iteration overlap.

The tell was the contradiction between A and E. If I had only run E, I would have concluded that virtual dispatch is free. If I had only run A, I would have had no idea the throughput win disappears under fencing.

3. Confounding arithmetic with dispatch

The first version of Experiment B compared four `EmaCross` legs against a mixed chain of four different strategies, and produced an incoherent result: monomorphic virtual appeared *slower* than polymorphic virtual.

It was. The mixed chain contained two cheap legs (spread gate, imbalance) and simply did less floating-point work. The clone types exist entirely to remove that confound.

The tell here was that the result was *impossible*, not merely surprising. Monomorphic dispatch cannot be slower than polymorphic dispatch of the same work. When a benchmark produces something that cannot be true, the benchmark is wrong — not the

hardware.

A fourth, smaller one

My first `inspect_asm.sh` grepped for `call *` and reported zero indirect branches in a function that clearly had one. GCC emits `jmp *rax` for a tail-called virtual function — same indirect branch, different mnemonic. Count both:

```
objdump -d yourfile.o | grep -E '(call|jmp)\s+\*'
```

CTSE: what it means, and when not to do this

The one-sentence conclusion

Inlining is not an optimisation — it is the enabling condition for every other optimisation. Constant folding, common-subexpression elimination across strategies, vectorisation, register allocation across the whole tick body. An opaque call blocks all of them at once. The `call *%rax` instruction itself is nearly free.

The mechanism, from the assembly probe

function	instructions	calls	spills
`probe_static_single`	77	0	0
`probe_virtual_single`	103	1 indirect	1
`probe_visible_final`	77	0	0
`probe_static_chain4`	421	3	15
`probe_virtual_chain4`	204	2	13

Note the opaque single-strategy version has **more** in-loop instructions (103 vs 77) even though the actual work moved elsewhere. That extra code is pure ABI overhead — marshalling arguments, saving live registers around an opaque callee.

Note also that `probe_static_chain4` has more instructions than `probe_virtual_chain4` (421 vs 204) for the obvious reason: the static version *contains* all four strategy bodies. **Instruction count is not a proxy for speed** — the static version runs at 82 cycles/tick and the virtual at 130.

And `probe_visible_final` is byte-identical to `probe_static_single`. Direct proof that devirtualisation fires and the vtable is not the problem.

When NOT to use this technique

Being able to say this is worth more than the benchmark.

- **Debug builds get slower.** With no inlining, CRTP is a chain of forwarding calls. Virtual is often faster at `-O0`.
- **Binary size and I-cache.** Every parameter set is a distinct instantiation. Eight strategies × six parameter sets is 48 copies competing for L1i. Past some point the win inverts.
- **Compile times.** Deep template composition is measured in minutes on real codebases. A real cost to a team's iteration speed.
- **No runtime configurability.** If strategies come from a config file, are hot-swapped, or loaded from a shared object, static dispatch is simply the wrong tool.
- **Template diagnostics.** Concepts help enormously, but a junior on the team will still pay for this.
- **You may not be call-bound at all.** 11 cycles/tick is ~4 ns. If the feed handler spends 800 ns in the kernel network stack, this is the wrong thing to optimise.

The hybrid that is usually right

Virtual at the outer boundary where you need flexibility; concrete types inside the hot leaf. And critically — **how you store the object decides everything**:

```
struct EngineA { Var1 a; Var2 b; };              // 0 indirect branches
struct EngineB { std::unique_ptr<Base> a, b; };  // 2 indirect branches
struct EngineC { Base& a; Base& b; };            // 2 indirect branches
```

Same classes, same `virtual`, same `final` in all three. Only the storage changed, and that alone decided whether the compiler could devirtualise. `EngineA` compiled to six instructions with both bodies inlined and interleaved — indistinguishable from CRTP.

So for an existing codebase the question is not "why virtual?" It is **"at the call sites that run per event, is the concrete type visible?"** That is checkable in five seconds:

```
objdump -d your_hot_object.o | grep -E '(call|jmp)\s+\*' | wc -l
```

LOB: the question

An exchange does not send you the order book. It sends a stream of **events**:

```
order #4471 added, 300 shares, buy, $99.98
order #4471 cancelled
order #3902 executed, 100 shares
order #5120 replaced by #5121, 200 shares, $99.99
```

You rebuild the book yourself, in memory, from those events. Millions of times per day, in nanoseconds, and **exactly right** — because your strategy trades on what it says.

So: how do you build that, how fast can it go, and how do you prove it is correct?

What "the book" has to answer

Question	Needed by
What is the best bid/ask right now?	Everything
How much is resting at each price?	Sizing, impact estimation
How many orders at that price?	Queue modelling
How many shares are ahead of *my* order?	Any passive strategy

That last one is why the per-price-level FIFO list exists. You can answer the first three with aggregate quantities alone — but then you cannot tell a passive strategy where it sits in the queue, which for market making is the entire edge.

Why correctness is harder here than it looks

An order book bug does not crash. The book stays plausible. The strategy keeps quoting. You find out from the P&L three weeks later.

That shapes the whole project. There are two implementations — a fast one and a deliberately naive one — and a differential test comparing them after **every single message**. The fast one is only trustworthy because the slow one exists.

Input and output

Input: a binary file in Nasdaq TotalView-ITCH 5.0 format — the real wire format. 60 MB, 2 million messages. Since a real Nasdaq capture cannot be downloaded in this environment, a generator produces a spec-conformant one; the parser handles real files unmodified.

Output: three programs. `verify` prints pass/fail. `replay` prints checksums and confirms determinism. `bench` prints timing tables.

LOB: itch.hpp — the wire format

Five things about ITCH that bite people, all handled explicitly.

1. Big-endian, every multi-byte field

x86 is little-endian, so every field needs a byte swap. `__builtin_bswap32` compiles to one `bswap` or `movbe` instruction — free. Forgetting it produces a book that is plausible and wrong.

2. Unaligned, always

Messages are packed with no padding and aligned to nothing.

```
// WRONG — undefined behaviour even on x86
std::uint32_t px = *reinterpret_cast<const std::uint32_t*>(p + 32);

// RIGHT — one mov after optimisation
[[gnu::always_inline]] inline std::uint32_t rd_be32(const std::uint8_t* p)
noexcept {
    std::uint32_t v;
    std::memcpy(&v, p, 4);
    return __builtin_bswap32(v);
}
```

The cast "works" on x86 right up until the compiler assumes alignment and vectorises accordingly. On ARM it faults outright. `memcpy` of a known small size is a single `mov` — zero cost, defined behaviour. There is no reason to take the risk.

3. Six-byte timestamps

48-bit big-endian nanoseconds since midnight. There is no 6-byte load instruction, and the obvious trick — read 8 bytes from offset 5 and mask — **runs past the end of the shortest message** (`Order Delete` is 19 bytes). So it composes a 4-byte and a 2-byte load:

```
inline std::uint64_t rd_be48(const std::uint8_t* p) noexcept {
    return (static_cast<std::uint64_t>(rd_be32(p)) << 16) | rd_be16(p + 4);
}
```

This is the kind of bug that passes every test until it hits the last message in a file, where the overread lands on an unmapped page.

4. Prices are integers with four implied decimals

$123.45 is stored as `1234500` . **They stay integers all the way through.**

Converting to `double` for storage is the single most common way to produce a book that disagrees with the exchange in the third decimal after a few million messages — and it is the main source of replay non-determinism, since float reassociation differs between `-O0` and `-O3`.

5. Framing length is authoritative

BinaryFILE prefixes each message with a 2-byte big-endian length. The per-type length table in the code is a **cross-check, not the parser's source of truth**:

```
const std::uint16_t expect = message_length(t);
if (expect != 0 && expect != len) h.on_length_mismatch(t, len, expect);
```

A mismatch is counted and reported rather than assumed away. Unknown message types are skipped by their framing length. That is what lets this code survive a spec revision adding a message type it has never seen.

mmap, not read()

The file is 1–10 GB for a real trading day, and the point of the exercise is deterministic replay, which means reading it many times. `mmap` lets the page cache do the work with no user-space copy.

`MADV_SEQUENTIAL` tells the kernel to read ahead aggressively and drop pages behind, which keeps a 10 GB replay from evicting everything else on the box. `MADV_HUGEPAGE` is deliberately *not* used here — file-backed mappings do not get transparent huge pages.

`prefault()` touches every page before timing starts. Without it, the first replay measures your disk.

The parse loop is templated on the handler

```
template <class Handler>
std::size_t parse(const std::uint8_t* data, std::size_t size, Handler& h,
...);
```

Same argument as the CTSE project, applied where it actually pays: this switch runs once per message, tens of millions of times per day. The handler's methods inline into the dispatch.

LOB: book.hpp — the structure

```
price ladder (flat array, one Level per tick)
   └─ Level { qty, count, head, tail }              16 B, 4 per cache line
        └─ intrusive doubly-linked FIFO of orders
             └─ Order { next, prev, qty, level|side }  16 B, 4 per cache line
                  (preallocated Pool, addressed by uint32 index)

order_ref (uint64) ──▶ OrderMap ──▶ pool index (uint32)
occupancy bitmap  ──▶ best bid / best ask in O(1)
```

operation	cost
add	O(1) — hash insert, pool alloc, tail append, bitmap set
cancel (partial)	O(1) — hash lookup, quantity decrement
delete	O(1) — lookup + erase, unlink, pool free, maybe bitmap reset
execute	O(1) — lookup, decrement, remove if filled
best bid/ask	O(1) — cached; recomputed via bitmap only when a level empties
top-K depth	O(K)
queue position	O(orders ahead at that level)

32-bit indices, not 64-bit pointers

```
struct Order {
    std::uint32_t next;        // index, not pointer
    std::uint32_t prev;
    std::uint32_t qty;
    std::uint32_t level_side;  // bit31 = side, bits 0..30 = ladder index
};
static_assert(sizeof(Order) == 16);
```

Halving every link field doubles the number of orders per cache line. For an intrusive list walked on every execution, that is the difference between one cache line and two.

Index **0 is the null sentinel**, so a zero-initialised structure is already "empty" and there is no separate invalid value to remember.

Side packed into the top bit

Keeps `Order` at exactly 16 bytes rather than 20-padded-to-24. A separate `bool side` member would cost 50% more memory per order for one bit of information.

```
const std::uint32_t idx = o.level_side & ~kAskBit;
const bool is_ask = (o.level_side & kAskBit) != 0;
```

The ladder is 65,536 levels, so 16 bits would do — but using bit 31 leaves room to grow the ladder without touching the layout.

Why a flat array instead of std::map

`std::map` is a red-black tree: every node separately allocated, ~48 bytes of overhead per node, and a lookup is 3–5 dependent pointer chases through memory the allocator scattered. The comparison at each node is cheap; **the cache miss before it is not.** Measured 3.4x slower.

The ladder sizing argument

65,536 levels × 16 bytes = 1 MB per side, for a book typically ~100 levels deep. That looks profligate and is not: **the allocation is virtual.**

Untouched pages are never faulted in, never occupy physical memory, never enter the TLB. The resident set is the few hundred ticks around the touch — a handful of 4 KB pages. You are trading address space, which is free on 64-bit, for pointer chasing, which is not.

The real cost is the bounded price range, handled explicitly rather than hidden:

- `base_price` and `tick` are constructor parameters. Default `tick = 100` (one cent in ITCH units), the quoting increment for any US equity ≥ $1.00 under Reg NMS Rule 612.
- Prices outside the window, or not on a tick boundary, are **rejected and counted** (`price_out_of_range`), never silently rounded. A book that quietly rounds is a book that quietly disagrees with the exchange.
- For a $3,000 name, pass a `base_price` near the open. Production would rebase the ladder on a large gap, or keep a secondary map for outliers.

The defensive clamp

```
const std::uint32_t take = shares < o.qty ? shares : o.qty;
```

A well-formed feed never over-executes an order. But a gap in the stream — a dropped multicast packet, a bad replay start point — produces exactly this, and underflowing a `uint32` turns a small inconsistency into a book with 4 billion shares on the bid.

LOB: pool.hpp — allocator and order map

Two rules the hot path obeys:

1. **No malloc. Ever.** Not `new`, not a `push_back` that might grow, not a `std::list` node. One unlucky allocation that walks into the kernel for a fresh page is a 10 µs outlier in a 100 ns budget, and it will land on the message you cared about most.
2. **No 64-bit pointers where a 32-bit index will do.**

The pool

Orders live in one contiguous, pre-faulted array. The free list is threaded **through the freed objects themselves**, so it costs no extra memory:

```
[[gnu::always_inline]] inline std::uint32_t alloc() noexcept {
    const std::uint32_t i = free_head_;
    if (i == kNullIdx) [[unlikely]] return kNullIdx;
    free_head_ = slots_[i].next;
    return i;
}
```

Allocation is a load and a store. Deallocation is two stores. Both branchless apart from the exhaustion check.

`reset()` threads slots in **ascending** order so the first allocations are sequential in memory and the prefetcher can help. `madvise(MADV_HUGEPAGE)` is requested — a pool of a few million entries is tens of MB, which at 4 KB pages burns thousands of dTLB entries.

The order map — four decisions

`std::unordered_map` is a bucket array of pointers to separately allocated nodes: two cache misses minimum per lookup, plus an allocation on every insert, which violates rule 1 outright. Measured **3.6x slower**.

Fibonacci hashing. Multiply by $2^{64}/\varphi$, take the **high** bits:

```
return static_cast<std::size_t>((k * 0x9E3779B97F4A7C15ULL) >> shift_);
```

ITCH order references are near-sequential. Taking the *low* bits of a sequential key would cluster catastrophically — consecutive keys landing in consecutive buckets, turning the table into a linear scan. The high bits of a multiplicative hash are well mixed even for sequential input.

Linear probing, not quadratic or double hashing. Probe sequences stay within a cache line most of the time, so a collision is usually free.

Backward-shift deletion, not tombstones. This is the one worth being able to defend. An order book is nothing but churn — the vast majority of orders are cancelled, never executed. With tombstones, deleted slots accumulate and the map slowly degrades into a linear scan over a trading day. Backward-shift pulls subsequent entries into the hole when their ideal slot allows it, preserving the "no empty slot inside a probe chain" invariant without tombstones.

Key 0 is the empty sentinel. ITCH order reference 0 does not occur.

The load-factor guard

```
static constexpr double kMaxLoad = 0.85;

[[gnu::always_inline]] inline void insert(std::uint64_t key, std::uint32_t
val) noexcept {
    if (full()) [[unlikely]] { ++overflows_; return; }
    ...
}
```

This guard exists because of a bug I hit. An open-addressing table with linear probing does **not** degrade gracefully when it fills — it *hangs*. There is no empty slot to terminate the probe, so the loop runs forever.

In a feed handler that failure mode is worse than a crash: the process is alive, the health check passes, and the book silently stops updating. The map now refuses to hang, counts overflows, and surfaces them through `BookStats::map_overflows`.

Entry layout

```
struct Entry {
    std::uint64_t key;
    std::uint32_t val;
    std::uint32_t _pad;
};
static_assert(sizeof(Entry) == 16, "4 entries per cache line");
```

Four entries per line means a linear probe sequence usually stays inside one cache fill.

LOB: bitmap.hpp — the occupancy index

The problem it solves

A flat ladder gives O(1) add and cancel at a known price. What it does not give you is **"what is the best bid now?"** after the current best level is depleted.

The naive answer walks down the ladder until a non-empty level is found. Usually that walk is 1–2 steps. Occasionally — after a sweep through thin book, or on a stock that quotes in wide increments — it is hundreds of steps.

And it happens at exactly the moment the market is moving and you least want a stall. **That is a latency spike correlated with market activity, which is the worst kind.**

The structure

For a ladder of $2^{16} = 65{,}536$ levels:

level	words	meaning
L0	1024	1 bit per price level
L1	16	1 bit per non-zero L0 word
L2	1	1 bit per non-zero L1 word

`find_highest_below` / `find_lowest_above` are then at most **3 dependent loads plus 3 bit-scans**, regardless of how far away the next level is. No loop, no data-dependent iteration count.

Total cost: 8 KB + 128 B + 8 B of bitmap for a 1 MB ladder, plus two extra stores on transition edges — only when a level's order count crosses zero, not on every message.

The subtlety in reset()

```
inline void reset(std::size_t i) noexcept {
    const std::size_t w0 = i >> 6;
    l0_[w0] &= ~bit(i);
    if (l0_[w0]) return;          // other levels in this block still
occupied
    const std::size_t w1 = w0 >> 6;
    l1_[w1] &= ~bit(w0);
    if (l1_[w1]) return;
    l2_[w1 >> 6] &= ~bit(w1);
}
```

Summary bits are cleared **only when the child word actually became zero**. Clearing unconditionally would be wrong — another level in the same 64-level block may still be occupied, and you would lose it.

Why the unit test is boundary-weighted

Three-level bit twiddling with inclusive and exclusive masks is exactly the kind of code that is 99% right, where the 1% is a word boundary firing once every few million messages. The test runs 400,000 random operations against a `std::set` oracle, with indices deliberately biased toward 0, 1, 62, 63, 64, 65, 4094, 4095, 4096, 4097, N−2, N−1.

What the measurement actually showed — and what it did not

arm	min	p50	p90	p99
linear ladder scan	30.89	32.03	34.75	35.58
3-level bitmap	29.29	30.54	35.34	38.04

About 5% on throughput, and the tail claim does not reproduce.

An earlier run showed the scan at p90/p99 of 59.7/72.9 against the bitmap's 43.1/44.7, and I wrote that up as a 1.63x tail win. Re-running reversed the ordering at p90/p99. That spread was VM noise, not signal.

The methodology error is worth stating plainly: those p90/p99 figures are quantiles over **5 whole-file replays**, so they describe how variable the machine was across five multi-second runs. They say nothing about the tail of an individual `delete` operation, which is what the bitmap targets. Aggregating 2 million operations into a mean and then taking a quantile over five of those means discards exactly the distribution you care about.

What *is* measured and does hold:

```
rescans triggered : 13367 (0.67% of messages)
ladder steps walked by the scan variant : 134773 (10.1 per rescan)
```

The mechanical argument stands — the scan's cost is data-dependent and grows with the gap to the next occupied level, while the bitmap is constant — but it is currently theory plus a step count, not a measured tail. Proving it needs per-operation latency distributions over `delete` / `execute` messages on a machine without other tenants.

Both arms still *maintain* the bitmap; only the search differs. So the 5% is a lower bound on the throughput effect.

LOB: refbook.hpp and the verification strategy

The oracle

`RefBook` is deliberately boring: `std::map` of price → `std::list` of orders, plus an `unordered_map` index. It allocates constantly, chases pointers everywhere, and is 3.4x slower.

That is fine. **Its job is not to be fast, it is to be so simple that a reviewer can convince themselves it is right by reading it.** 100 lines, no cleverness.

Every optimisation in `book.hpp` is then justified by a differential test rather than by assertion.

Shared acceptance rules — the subtle requirement

The two implementations must share their **acceptance rules**, or the diff is meaningless:

```
bool acceptable(std::uint32_t price) const {
    if (price < base_) return false;
    const std::uint32_t d = price - base_;
    if (d % tick_ != 0) return false;          // sub-tick prices rejected
    return (d / tick_) < max_levels_;
}
```

If the fast book rejects a sub-penny price and the reference accepts it, every subsequent comparison fails for a reason that has nothing to do with the data structures. You would spend a day debugging a divergence that is an artefact of your test harness.

What is checked, and how often

check	frequency
Bitmap vs `std::set` oracle	400,000 ops, boundary-weighted
Top of book	**every message**
Top-20 depth, both sides	**every message**
Live order count	**every message**
Full-ladder checksum	every 25,000 messages
Queue position (shares ahead)	at checkpoints, sampled orders

The queue-position check matters because it is invisible to an aggregate L2 comparison. Two books can agree on every price and quantity while disagreeing about who is first in the queue — and for a passive strategy that difference is the entire edge.

Deterministic replay

Two independent passes over the same bytes must produce identical checkpoint checksums, and those must match a committed golden file.

This catches the four usual sources of replay drift:

1. **Floating-point reassociation** under different optimisation levels. Avoided entirely here — prices never become floats.
2. **Unordered-container iteration order** leaking into output.
3. **Uninitialised padding** entering a checksum.
4. **Pointer values influencing behaviour** (a hash keyed on an address). ASLR then makes every run different.

Checkpoints every 100,000 messages, **not just at end of day**. An end-of-day-only checksum tells you *that* something diverged and nothing about *where*; a checkpoint series bisects it for you.

The golden file is committed. It is the cheapest possible regression test: any refactor that changes book semantics — even one that keeps the top of book identical — fails the instant it changes any level's quantity anywhere in the ladder.

Results

```
[ OK ] HierBitmap vs std::set   400000 ops, boundary-weighted
[ OK ] top-of-book             2000004 messages, checked every message
[ OK ] top-20 depth both sides 2000004 messages, checked every message
[ OK ] live order count        2000004 messages, checked every message
[ OK ] full book checksum      78 checkpoints
[ OK ] queue position (FIFO)   72682 order comparisons

anomaly counters (all 0 on a clean feed)
  unknown order ref  : 0
  price out of range : 0
  pool exhausted     : 0
  length mismatches  : 0

REPLAY IS DETERMINISTIC.   final checksum: 0x934eb20eaa23c4f9
```

LOB: gen_itch.cpp — the market model

Why generate instead of downloading

Nasdaq publishes free daily TotalView-ITCH samples and the parser consumes them unmodified. A generated file adds three things a downloaded one cannot:

1. **Determinism.** A fixed seed produces byte-identical input, which is what makes "replay twice, get the same checksum" a statement about the book rather than about your disk.
2. **Adversarial control.** Real data does not contain a 500-deep price level or a sweep emptying 400 levels on demand. Those are exactly the cases that break a book which finds its best price by scanning.
3. **No multi-GB download in CI.**

The mistake worth documenting

The first version kept live orders in a flat `vector` and cleared "crossed" orders by sampling a few at random each iteration.

That does not work, and the way it fails is instructive. The mid random-walks away. Stale orders on the wrong side accumulate faster than random sampling removes them. Within a million messages **the best bid sat 2,000 ticks above the best ask.**

A persistently crossed book is not a market. And worse: the touch level never empties, so best-price recomputation never fires, and the occupancy bitmap — the central data structure of the project — goes entirely untested by a file that looks fine.

The differential verifier passed the whole time. Both books agreed perfectly, on nonsense.

What replaced it

A real price-indexed model: `price → FIFO of order references`, per side. When the mid moves, every level now through the market is cleared explicitly, order by order, as executions and cancels:

```
auto clear_through_market = [&] {
    for (int guard = 0; guard < 128; ++guard) {
        const std::uint32_t bb = model.best_bid();
        const std::uint32_t ba = model.best_ask();
        const bool bid_through = bb != 0 && static_cast<double>(bb) > mid;
        const bool ask_through = ba != 0 && static_cast<double>(ba) < mid;
        if (!bid_through && !ask_through) break;
        // ... emit execute (55%) or delete (45%) for every order at that
level
    }
};
```

That is the actual mechanism in a real market: orders through the market are either taken by aggressive flow or pulled by their owner. It produces a book that is never crossed, has a touch that churns constantly, and exercises the code paths the project exists to test.

Rescans went from **0.00% to 0.67%** of messages.

Design details that matter

Exponentially distributed depth. `depth = -log(u) * 8` means the book is dense at the touch and thins out — which makes best-price recomputation cheap most of the time and expensive exactly when the market moves. A uniform distribution would make every rescan equally cheap and hide the effect.

Two adversarial events, deliberately rare:

- *Liquidity sweep* — empties a long contiguous run of levels in one burst. The worst case for a scanning best-price search.
- *Deep-level pileup* — 500 orders at one price, stressing the per-level FIFO and the queue-position walk.

Every emitted message is semantically valid. The generator maintains its own live-order set, so it never deletes an order that does not exist, never executes more than the resting size, never replaces a dead reference. A parser tested only against malformed input learns nothing.

The resulting file

```
messages         : 2,000,000     bytes: 60.4 MB
add              : 45.0%        delete : 29.9%
cancel (partial) :  4.3%        execute: 16.4%
replace          :  3.6%        trade  :  0.7%
resting at close : 5,039 orders across 90 bid / 102 ask levels
cleared through  : 493,268 orders taken out by market moves
stress events    : 145 sweeps, 215 deep-level pileups
```

LOB: the experiments

Same hardware as CTSE. 2,000,004 messages / 60.4 MB, 5 interleaved rounds.

One structural decision: the ITCH file is parsed **once** into a flat array of normalised operations, and the book benchmarks replay that array. Otherwise every book measurement silently includes the parser — a fixed cost shared by all arms, which would compress every ratio toward 1.0 and hide the effect being measured. Parsing is benchmarked separately.

A — decode vs decode + book

arm	ns/msg	cycles/msg	M msg/s
ITCH decode only	8.07	22.6	124
ITCH decode + book update	38.78	108.6	26

Parsing is **21% of end-to-end cost**.

B — flat ladder vs std::map

arm	ns/msg	vs
`std::map` + `std::list` + `unordered_map`	115.18	1.00x
flat ladder + pool + open addressing	33.84	**3.40x**

Same message stream, semantics proven identical by `make verify`.

C — best price: bitmap vs ladder scan

Covered in the bitmap section. **~5% on throughput**; the tail claim was withdrawn as unreproducible.

D — order map, isolated

arm	ns/msg	vs
`std::unordered_map<uint64,uint32>`	84.76	1.00x
open addressing + Fibonacci hash	23.40	**3.62x**

E — per-message-type latency

Fenced per operation, overhead subtracted. Cycles.

message	count	min	p50	p90	p99	p99.9
add	910,698	36	260	380	1050	1361
execute	296,654	20	114	316	798	1284
cancel	79,851	14	160	372	828	1412
delete	633,289	24	156	346	848	1324
replace	66,529	90	356	614	1262	1645

`replace` costs roughly `delete` + `add` because it **is** delete plus add — and it loses queue priority for the same reason. A strategy that wants to keep its queue position reduces size with a partial cancel instead.

Absolute numbers are inflated by fencing, as in CTSE Experiment E. Read the *shape* here and the *magnitudes* from A and B.

F — query cost on a warm book

5,039 resting orders across 90 bid / 102 ask levels. Cycles.

query	min	p50	p99
top of book	2	4	8
L2 depth, 10 levels	146	148	320
L2 depth, 50 levels	710	716	824
full-ladder checksum	15,226	15,244	27,005

Top of book is 4 cycles because it is two cached indices and two loads. **A book that is fast to build but slow to interrogate is not useful** — the strategy reads it far more often than the feed writes it.

G — capacity sizing

The single largest effect in the project, and it is not an algorithm.

configuration	ns/msg	alloc MB	vs best
pool 2^{18}, map 2^{18}	**29.96**	8	1.00x
pool 2^{18}, map 2^{20}	74.35	20	2.48x
pool 2^{18}, map 2^{22}	88.08	68	2.94x
pool 2^{21}, map 2^{18}	34.27	36	1.14x
pool 2^{21}, map 2^{22}	85.34	96	2.85x

Peak live orders in this file: **30,008**. Sizing the map at 4M slots "to be safe" costs **2.9x throughput**.

The map dominates and the pool barely matters, and the reason is **access pattern, not size**. A hash spreads lookups uniformly across the entire table, so the allocation *is* the working set — a 64 MB map means a cache miss plus a TLB miss on every lookup. The pool is filled densely from slot 0 upward, so an oversized pool costs address space and essentially no time.

The corollary worth internalising: size hash tables tightly and measure; be generous with anything accessed sequentially. The instinct to over-provision "just in case" is correct for the pool and expensive for the map.

LOB: the four failures

These are the most valuable part of the repo. Anyone can quote a 3.4x number.

1. A green test suite on impossible input

Covered in the generator section, but worth stating as a principle.

The differential verifier passed every check — top of book, top-20 depth, live order count, queue position, full checksums — on a book where **the best bid sat 2,000 ticks above the best ask.** Both implementations were faithfully modelling nonsense.

The only tell was a performance counter reading `best-price rescans: 0.00% of messages`. The touch never emptied, so the bitmap was never exercised.

A passing differential test proves two implementations agree. It says nothing about whether your input exercises the paths you built. Instrument the code paths you care about and check the counters, not just the assertions.

2. A hash map that hung instead of failing

`make bench` produced no output and did not crash. The process sat at 100% CPU.

Linear probing terminates by finding an empty slot. When the table is full, there isn't one, and `insert` loops forever. My benchmark harness only erased on delete and replace — not on executes that take an order to zero — so it accumulated ~277k entries in a 262,144-slot table.

In a feed handler this failure mode is **worse than a crash**: the process is alive, the health check passes, and the book silently stops updating. Fixed with an explicit 0.85 load ceiling and an overflow counter.

The general principle: **a data structure should fail loudly at its limit, not degrade into an infinite loop.** Ask of every structure you write: what happens when it is full?

3. A tail-latency result that was noise

I reported the bitmap as a 1.63x win at p99 based on one benchmark run. It did not reproduce — a later run reversed the ordering.

The p90/p99 columns in that experiment are quantiles over five whole-file replays, so they measure how noisy the VM was across five multi-second runs, not the tail of any individual operation. Aggregating 2 million operations into a mean and then taking a quantile across five of those means throws away the distribution you are trying to see.

The corrected write-up is in the bitmap section. Everything that survived re-running is reported as measured; this one did not, and it is **labelled as a mistake rather than deleted**.

The lesson is about publishing, not measurement: *a single run is not a result.* Re-run before you write it down.

4. My own defaults were the slowest configuration

The original defaults were `pool = 2²¹, map = 2²²` — chosen by "that seems safely large." That is the 2.85x row in experiment G.

Fixing two constants beat every algorithmic choice in the project.

This is the one to lead with in an interview, because it inverts the expected story. The candidate who says "I used a flat array instead of a tree and got 3.4x" sounds like they read a blog post. The candidate who says "my biggest win was making a data structure smaller, and here is the cache/TLB reason" has actually profiled something.

Rebuilding these yourself

The method that works

Close the reference before writing each function. Read it, understand it, close the tab, write it, *then* diff. If you type with the reference open beside you, your hands learn it and you do not. The diff is where you find out whether you understood `static_cast<Derived*>(this)` or just transcribed it.

Build in dependency order and run something after each file. Do not move on until the checkpoint passes.

CTSE build order

#	file	checkpoint
1	`types.hpp`	a `main` printing `sizeof(Tick)`, `sizeof(Signal)`; assert alignment
2	`clock.hpp`	print calibrated TSC GHz, sanity-check against `lscpu`
3	`bench.hpp`	feed `Stats::from` a known array, verify p50/p99 by hand
4	`indicators.hpp`	run an EMA over 100 points, check against a 5-line Python script
5	`cores.hpp`	print signals for 20 ticks, confirm they are not all zero
6	`static_engine.hpp`	chain of 2 compiles and runs
7	`virtual_engine.hpp` + `virtual_impl.cpp`	both factories construct
8	`data.hpp`	print 10 ticks; spreads positive, prices sane
9	**`verify_main.cpp`**	**hard gate — no benchmark code until this passes**
10	`bench_main.cpp`	one experiment at a time
11	`asm_probe.cpp` + `inspect_asm.sh`	call counts match expectations

The value is concentrated at step 9. If your `verify` fails, you have found a real bug in something you wrote, and debugging it is worth more than the other ten steps combined.

LOB build order

#	file	checkpoint
1	`itch.hpp` byte readers	round-trip: `wr_be32` then `rd_be32` returns the input
2	`bitmap.hpp`	**the `std::set` differential test** — do this before the book
3	`pool.hpp` pool	alloc all, free all, alloc again; high-water correct
4	`pool.hpp` map	insert/find/erase 100k keys vs `std::unordered_map`
5	`refbook.hpp`	hand-feed 10 messages, print the book, check by eye
6	`book.hpp`	same 10 messages, same output
7	`gen_itch.cpp`	**check `rescans > 0`**, and that the book is never crossed
8	`verify_main.cpp`	the differential gate
9	`replay_main.cpp`	two passes, same checksum
10	`bench_main.cpp`	one experiment at a time

Step 2 before step 6 matters. A bitmap bug found through the book is painful to localise; found directly against `std::set`, it takes minutes.

Step 7 is the one I got wrong. **Check the counter, not just that it compiles.**

Deliberately diverge

A verbatim copy is hard to defend and boring to build. Three concrete suggestions:

- **Swap the strategies for ones from your domain.** Index F&O gives you an OI-change signal, a futures-basis deviation, an IV-rank filter. Same shape, your semantics, and you can talk about them fluently.
- **Add the `std::variant` + `std::visit` arm to CTSE.** Closed type set, no vtable, objects stored inline, a jump table instead of an indirect call — often the right middle ground. It is a genuine result nobody handed you, and it directly answers the "what about our variable graph?" question.

- **Break something on purpose.** Change one template parameter in the virtual arm so the arms compute different things. Confirm `verify` catches it and names the exact tick. A test you have watched fail is a test you trust.

Write your own README from your own runs

Do not port the numbers in this guide — you will be on different hardware and they will be wrong. The traps carry over as **checks to perform**, not as conclusions to restate. Re-run `make icf` yourself; if your GCC version behaves differently, that is your finding.

The test for whether it is yours

A week after you finish, delete the repo and rewrite `types.hpp` and `static_engine.hpp` from memory. If you can, it is yours.

On provenance

"I built this after working through a reference design, then extended it with X and Y" is completely normal and nobody will blink. What would hurt is presenting it as from-scratch original work and then not being able to explain why `sink()` uses `"r,m"` instead of `"m"`.

The bar is not "did I type it." It is **"can I defend every choice under questioning."**

Interview framing

Lead with the **finding**, not the project. "I built a benchmark" is not a conversation. "I measured virtual dispatch and found the vtable isn't the cost" is.

"Why is virtual dispatch slow?"

Mostly it isn't the `call`. Ranked by measured contribution:

1. **The optimisation barrier** — no inlining means no constant propagation, no CSE across legs, no vectorisation, plus ABI spills at each call site. 70–90% of the delta.
2. **Indirect branch misprediction** — ~14 cycles per call, *but only when the type sequence is unpredictable*. A fixed 0,1,2,3 cycle costs nothing.
3. **Memory layout** from `vector<unique_ptr<>>` — measured at **zero** in this workload, because four legs fit in L1.

"When would you use virtual anyway?"

Runtime-configurable strategy sets, plugin boundaries, anything crossing an ABI, every cold path. Also: mark leaf classes `final` and keep them header-visible and the compiler devirtualises for free — measured identical to CRTP.

And the sharper version: **storage decides, not the keyword.** Hold concrete members by value and you get CRTP performance with a virtual interface. Hold them behind `unique_ptr<Base>` and you pay in full.

"How do you know your benchmark is correct?"

Shared arithmetic cores, a bit-identical equivalence gate that must pass before any timing runs, interleaved arms, min-based reporting with an explicit noise caveat — and a documented case (`make icf`) where a default compiler flag silently erased a real 59-cycle effect.

"How do you build a fast order book?"

Flat ladder instead of a tree (3.40x), open-addressing map instead of `unordered_map` (3.62x), 32-bit indices instead of pointers, no allocation on the hot path — and, measured largest of all, **size the hash table tightly, because a hash makes the allocation the working set** (2.94x).

"How do you know the book is correct?"

Differential testing against a naive `std::map` reference after every message, including **queue position**, not just aggregate depth. Plus deterministic replay against committed checkpoint checksums, so any semantic change fails immediately and bisects itself.

"Have you worked with market data?"

Big-endian unaligned reads via `memcpy`. 48-bit timestamps composed from 4+2 byte loads because an 8-byte read overruns the shortest message. Integer prices with four implied decimals, never converted to float. Why replace loses queue priority. How you would prove your book matches the exchange.

"What went wrong while you built it?"

This is the question that separates you, and you have seven real answers. The two best:

- **A fully green test suite running against a book that was physically impossible** — crossed by 2,000 ticks. Caught by a performance counter reading 0.00%, not by any assertion.
- **A published 1.63x tail result that turned out to be VM noise.** Withdrawn and labelled in the README rather than quietly deleted, with the methodology error explained.

The second one is uncomfortable to volunteer and worth more than any speedup in either repo. Most candidates present only wins.

If asked about your firm's architecture

You now have a measured answer rather than an opinion:

> Virtual costs roughly 12–30 cycles per opaque call, and only where the compiler cannot see the concrete type. At one dispatch per strategy, per variable, and per execution — about a dozen per tick — that is well under 100 ns against a budget dominated by the network path. And it buys runtime configurability, which templates cannot provide at all.

That is a better answer than either "virtual is slow" or "virtual is fine." It is the trade-off with numbers attached.